# train_resnet

February 21, 2026

[27]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report,␣
↪ConfusionMatrixDisplay, precision_recall_fscore_support
import tensorflow as tf
import tensorflow_addons as tfa
#import wandb
#from wandb.keras import WandbMetricsLogger
from models import Xception, ResNet50
sns.set_style("darkgrid")
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
```

[28]:
```
df = pd.read_csv('../data/metadata - standardized.csv')
df = pd.DataFrame(df[['names','types']])
```

[29]:
```
df.info()
```

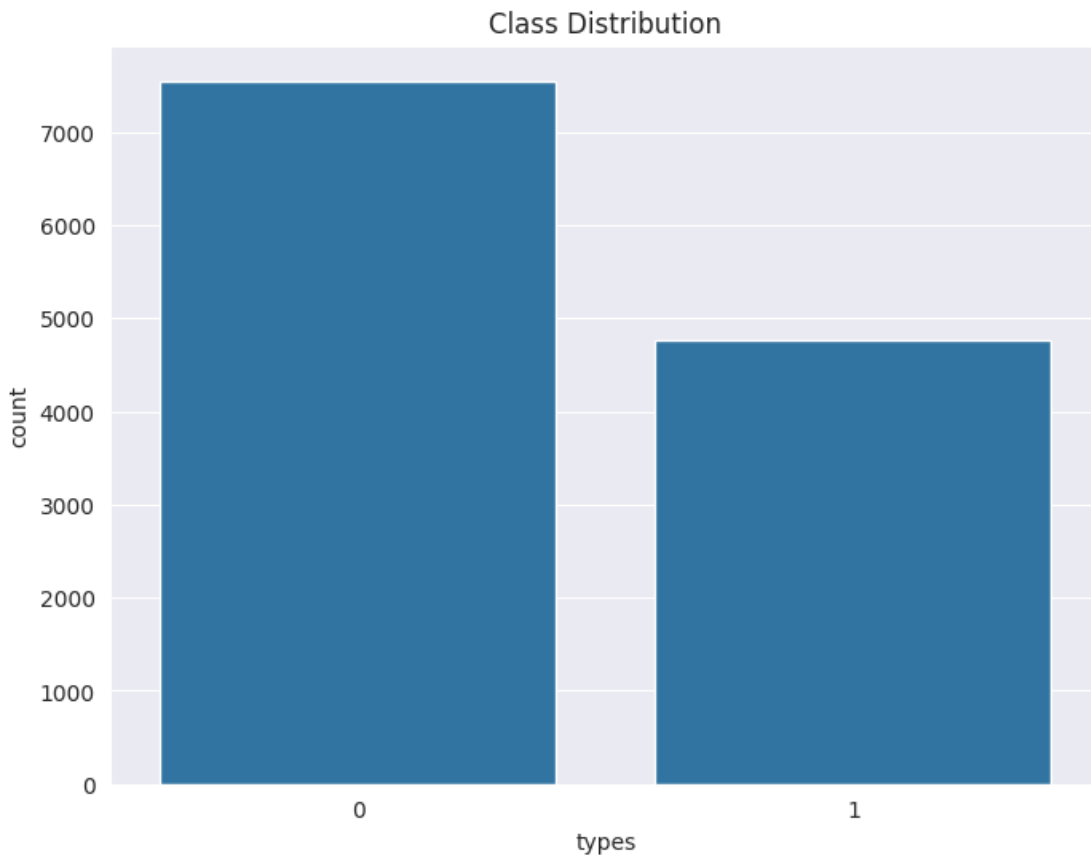
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12449 entries, 0 to 12448
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   names   12449 non-null  object
 1   types   12449 non-null  int64
dtypes: int64(1), object(1)
memory usage: 194.6+ KB
```

[30]:
```
df.isnull().sum()
```

[30]:
```
names    0
types    0
dtype: int64
```

[31]:
```
df = df[df['types'] != -1]
```

[32]:
```
plt.figure(figsize=(8,6))
sns.countplot(x=df['types'])
plt.title("Class Distribution")
plt.show()
```




[33]:
```
image_directory = r'..\data\full-fundus\full-fundus'
image_size = 256
seed = 12
batch_size = 64

def get_path(name):
    return image_directory + '\\' + name + '.png'

df['path'] = df['names'].apply(get_path)
df.drop('names', axis=1, inplace=True)
```

[34]:
```
train_data, val_data = train_test_split(df, test_size=0.2, random_state=seed)
train_data, test_data = train_test_split(train_data, test_size=0.1,␣
↪random_state=seed)
```

[35]:
```python
train_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8866 entries, 3787 to 18
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   types   8866 non-null   int64
 1   path    8866 non-null   object
dtypes: int64(1), object(1)
memory usage: 207.8+ KB
```

[36]:
```python
import os

BASE_DIR = "../data/full-fundus/full-fundus"

def fix_path(p):
    # Replace Windows slashes
    p = p.replace("\\", "/")

    # Remove everything before last occurrence of full-fundus
    filename = p.split("full-fundus/")[-1]

    # Build correct absolute path
    return os.path.join(BASE_DIR, filename)

train_data["path"] = train_data["path"].apply(fix_path)
val_data["path"] = val_data["path"].apply(fix_path)
test_data["path"] = test_data["path"].apply(fix_path)
```

[37]:
```python
val_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2464 entries, 446 to 10671
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   types   2464 non-null   int64
 1   path    2464 non-null   object
dtypes: int64(1), object(1)
memory usage: 57.8+ KB
```

[38]:
```python
test_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 986 entries, 7308 to 5484
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
```

```
 0   types   986 non-null    int64
 1   path    986 non-null    object
dtypes: int64(1), object(1)
memory usage: 23.1+ KB
```

[39]:
```python
class ImageGenerator(tf.keras.utils.Sequence):
    """
    Custom Image data generator class from Image paths and labels
    """
    def __init__(self, data: pd.DataFrame, batch_size: int=batch_size,␣
↪image_size: tuple=image_size, balance: bool=False, augment: bool=False):
        if balance:
            self.data = self.__balance__(data)
        else:
            self.data = data
        self.labels = self.data['types'].values
        self.image_path = self.data['path'].values
        self.batch_size = batch_size
        self.image_size = (image_size, image_size)
        self.augment = augment
        self.augment_pipe = tf.keras.Sequential([
            tf.keras.layers.experimental.preprocessing.
↪RandomFlip("horizontal_and_vertical"),
            tf.keras.layers.experimental.preprocessing.RandomZoom(0.3),
            tf.keras.layers.experimental.preprocessing.RandomContrast(0.2),
            tf.keras.layers.experimental.preprocessing.RandomRotation(0.1),
            tf.keras.layers.experimental.preprocessing.RandomTranslation(0.2, 0.
↪2)
        ])

    def __augment__(self, image):
        if np.random.rand() < 0:
            image = self.augment_pipe(image)
        return image

    def __balance__(self, data):
        data_pos, data_neg = data[data['types'] == 1], data[data['types'] == 0]
        data_neg = data_neg.sample(len(data_pos), replace=False,␣
↪random_state=seed)
        data = pd.concat([data_pos, data_neg])
        data = data.sample(frac=1)
        return data

    def __len__(self):
        return np.math.ceil(len(self.image_path) / self.batch_size)

    def __get_image__(self, image_path):
```

```
        image = tf.keras.preprocessing.image.load_img(image_path,␣
↪target_size=self.image_size)
        image = tf.keras.preprocessing.image.img_to_array(image)
        return image

    def __getitem__(self, index):

        batch_images = self.image_path[index * self.batch_size : (index + 1) *␣
↪self.batch_size]
        batch_images = np.array([self.__augment__(self.__get_image__(path)) for␣
↪path in batch_images]) / 255.0
        batch_labels = self.labels[index * self.batch_size : (index + 1) * self.
↪batch_size]
        return batch_images, batch_labels
```

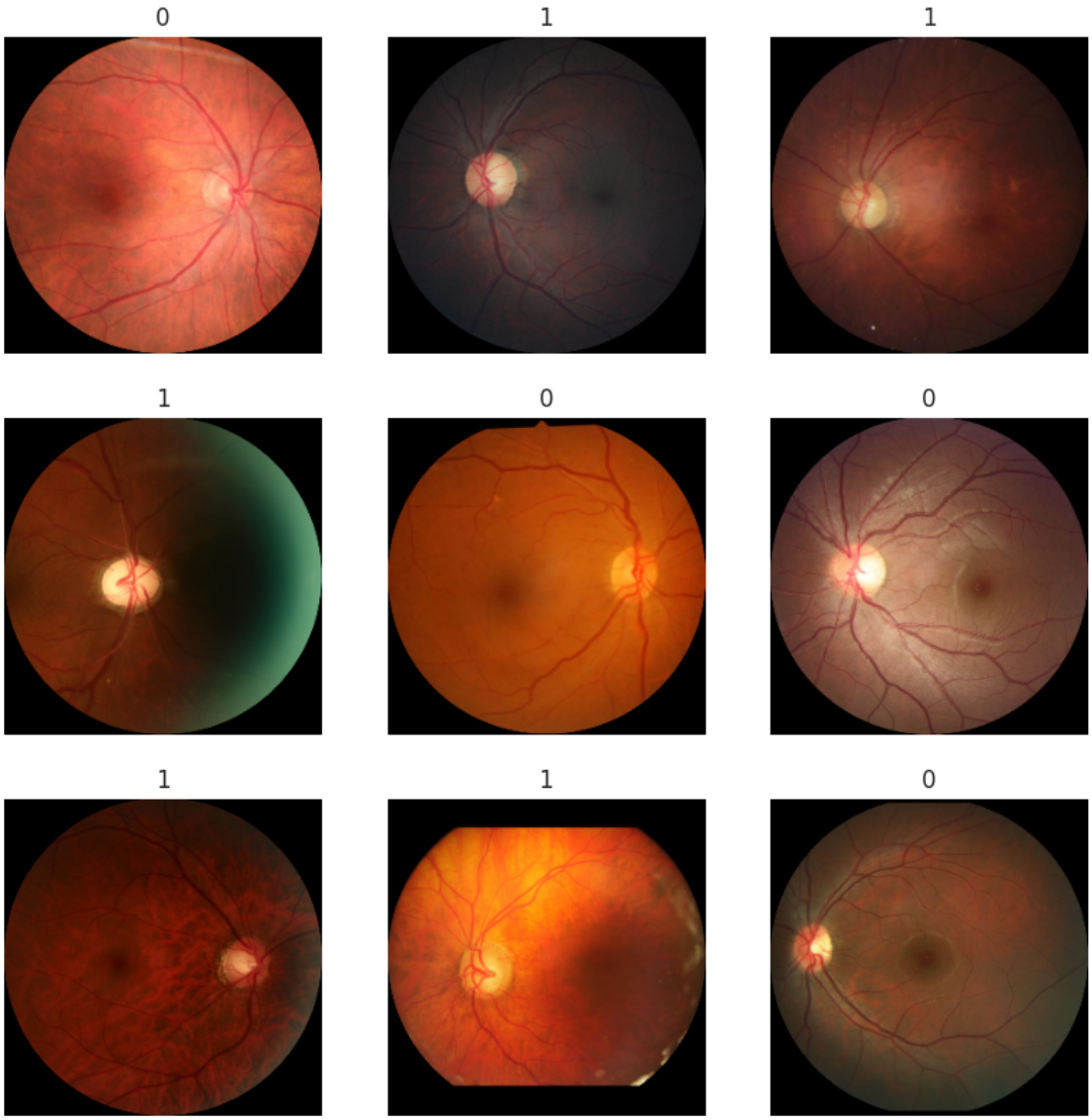
[40]:
```
train_dataset = ImageGenerator(train_data, balance=True, augment=True)
val_dataset = ImageGenerator(val_data, balance=False, augment=False)
test_dataset = ImageGenerator(test_data, balance=False, augment=False)
```

[41]:
```
print(train_data['path'].head())
```

```
3787    ../data/full-fundus/full-fundus/OIA-ODIR-TRAIN…
1241    ../data/full-fundus/full-fundus/OIA-ODIR-TEST-…
1135    ../data/full-fundus/full-fundus/OIA-ODIR-TEST-…
7863        ../data/full-fundus/full-fundus/G1020-592.png
7108    ../data/full-fundus/full-fundus/REFUGE1-val-23…
Name: path, dtype: object
```

[42]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
im,l = train_dataset.__getitem__(np.random.randint(0, len(train_dataset)))
for i in range(9):
  ax = plt.subplot(3, 3, i+1)
  plt.imshow(im[i])
  plt.title(l[i])
  plt.axis("off")
```

```
/tmp/ipykernel_732028/4227412241.py:36: DeprecationWarning: `np.math` is a
deprecated alias for the standard library `math` module (Deprecated Numpy 1.25).
Replace usages of `np.math` with `math`
  return np.math.ceil(len(self.image_path) / self.batch_size)
```




## 0.1 ResNet50 Model

### 0.1.1 Initializing

**Hyperparameters**

[43]:
```
learning_rate = 0.001
weight_decay = 0.0006
beta = 0.5
num_epochs = 50
checkpoint_filepath_res = r'F:\Glaucoma\saved_model\resnet\weights.h5'
```

**Wandb**

[44]:
```
# wandb.init(
#     project="Glaucoma Detection",
#     name = 'ResNet50',

#     # track hyperparameters and run metadata
#     config={
#         "Dataset": "Glaucoma",
#         "Models": "ResNet50",
#         "learning_rate": learning_rate,
#         "weight_decay": weight_decay,
#         "metrics": ["accuracy"],
#     }
# )
```

**Trainer**

[45]:
```
def run_training(model, load_checkpoint=True):
    optimizer = tfa.optimizers.AdamW(
        learning_rate=learning_rate,
        weight_decay=weight_decay
    )

    model.compile(
        optimizer=optimizer,
        loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
        metrics=[
           'accuracy',
        ],
    )

    checkpoint_callback = tf.keras.callbacks.ModelCheckpoint(
        checkpoint_filepath_res,
        monitor="val_loss",
        save_best_only=True,
        save_weights_only=True,
    )

    early_stopping_callback = tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True,
    )

    if load_checkpoint:
        model.load_weights(checkpoint_filepath_res)

    history = model.fit(
        train_dataset,
```

```
        epochs=num_epochs,
        validation_data=val_dataset,
        callbacks=[checkpoint_callback,
                   early_stopping_callback,
                #    WandbMetricsLogger(log_freq='batch')
                   ],
    )
    return history
```

**Evaluator**

[46]:
```
def run_evalutation(model, dataset):
    optimizer = tfa.optimizers.AdamW(
        learning_rate=learning_rate,
        weight_decay=weight_decay,
    )

    model.compile(
        optimizer=optimizer,
        loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
        metrics=[
           'accuracy',
        ],
    )
    model.load_weights(checkpoint_filepath_res)
    _, accuracy = model.evaluate(dataset)
    predictions = model.predict(dataset)
    predictions = tf.where(predictions < 0.5, 0, 1)
    report  = classification_report(dataset.labels, predictions)
    return accuracy, report
```

### 0.1.2 Building the Model

[47]:
```
tf.keras.backend.clear_session()
res_classifier = ResNet50(input_shape=(image_size, image_size, 3),␣
 ↪num_classes=1).build_model()
```

### 0.1.3 Training

[48]:
```
history = run_training(res_classifier, load_checkpoint=False)
```

```
/tmp/ipykernel_732028/4227412241.py:36: DeprecationWarning: `np.math` is a
deprecated alias for the standard library `math` module (Deprecated Numpy 1.25).
Replace usages of `np.math` with `math`
  return np.math.ceil(len(self.image_path) / self.batch_size)
```

```
Epoch 1/50
```

```
2026-02-20 06:26:51.479678: I
```

```
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:454] Loaded cuDNN
version 8907
108/108 [==============================] - 85s 676ms/step - loss: 0.5010 -
accuracy: 0.7754 - val_loss: 0.7251 - val_accuracy: 0.3799
Epoch 2/50
108/108 [==============================] - 62s 577ms/step - loss: 0.3662 -
accuracy: 0.8374 - val_loss: 0.6928 - val_accuracy: 0.6201
Epoch 3/50
108/108 [==============================] - 63s 579ms/step - loss: 0.3282 -
accuracy: 0.8542 - val_loss: 0.6927 - val_accuracy: 0.6201
Epoch 4/50
108/108 [==============================] - 62s 576ms/step - loss: 0.2952 -
accuracy: 0.8693 - val_loss: 0.6927 - val_accuracy: 0.6201
Epoch 5/50
108/108 [==============================] - 65s 603ms/step - loss: 0.2855 -
accuracy: 0.8751 - val_loss: 0.6796 - val_accuracy: 0.6201
Epoch 6/50
108/108 [==============================] - 62s 575ms/step - loss: 0.2759 -
accuracy: 0.8745 - val_loss: 0.6949 - val_accuracy: 0.3815
Epoch 7/50
108/108 [==============================] - 62s 576ms/step - loss: 0.2492 -
accuracy: 0.8899 - val_loss: 0.6799 - val_accuracy: 0.6201
Epoch 8/50
108/108 [==============================] - 63s 581ms/step - loss: 0.2344 -
accuracy: 0.8990 - val_loss: 0.6957 - val_accuracy: 0.4010
Epoch 9/50
108/108 [==============================] - 62s 575ms/step - loss: 0.2248 -
accuracy: 0.8992 - val_loss: 0.6865 - val_accuracy: 0.6449
Epoch 10/50
108/108 [==============================] - 65s 600ms/step - loss: 0.2206 -
accuracy: 0.9050 - val_loss: 0.6138 - val_accuracy: 0.6802
Epoch 11/50
108/108 [==============================] - 61s 565ms/step - loss: 0.2209 -
accuracy: 0.9072 - val_loss: 0.5649 - val_accuracy: 0.7581
Epoch 12/50
108/108 [==============================] - 62s 567ms/step - loss: 0.1866 -
accuracy: 0.9248 - val_loss: 0.4359 - val_accuracy: 0.8076
Epoch 13/50
108/108 [==============================] - 63s 582ms/step - loss: 0.1969 -
accuracy: 0.9176 - val_loss: 0.5428 - val_accuracy: 0.7695
Epoch 14/50
108/108 [==============================] - 62s 575ms/step - loss: 0.1818 -
accuracy: 0.9220 - val_loss: 0.7137 - val_accuracy: 0.5130
Epoch 15/50
108/108 [==============================] - 61s 566ms/step - loss: 0.1747 -
accuracy: 0.9269 - val_loss: 0.5345 - val_accuracy: 0.7642
Epoch 16/50
```

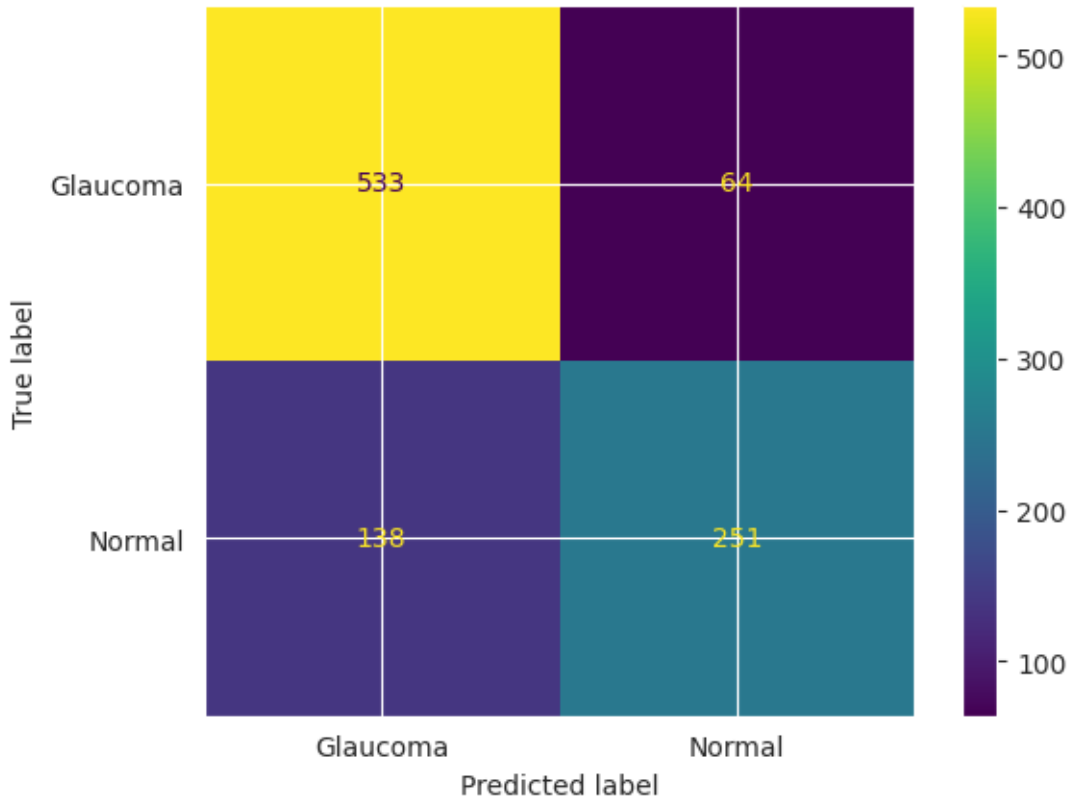
```
108/108 [==============================] - 62s 573ms/step - loss: 0.1496 -
accuracy: 0.9404 - val_loss: 0.5226 - val_accuracy: 0.7979
Epoch 17/50
108/108 [==============================] - 63s 581ms/step - loss: 0.1729 -
accuracy: 0.9306 - val_loss: 0.5106 - val_accuracy: 0.7654
```

### 0.1.4 Evaluation

[49]:
```
accuracy, report = run_evalutation(res_classifier, test_dataset)
pred_label = res_classifier.predict(test_dataset) > 0.5
precision,recall,f1_score,_ =  precision_recall_fscore_support(test_dataset.
 ↪labels, pred_label, average='macro')
cm = confusion_matrix(test_dataset.labels, pred_label)
print("Accuracy on test dataset: ", accuracy)
print("Classification report:\n", report)
print("Confusion matrix:\n")
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Glaucoma',␣
 ↪'Normal'])
disp.plot()
```

```
/tmp/ipykernel_732028/4227412241.py:36: DeprecationWarning: `np.math` is a
deprecated alias for the standard library `math` module (Deprecated Numpy 1.25).
Replace usages of `np.math` with `math`
  return np.math.ceil(len(self.image_path) / self.batch_size)

16/16 [==============================] - 10s 514ms/step - loss: 0.4161 -
accuracy: 0.7951
16/16 [==============================] - 8s 419ms/step

/tmp/ipykernel_732028/4227412241.py:36: DeprecationWarning: `np.math` is a
deprecated alias for the standard library `math` module (Deprecated Numpy 1.25).
Replace usages of `np.math` with `math`
  return np.math.ceil(len(self.image_path) / self.batch_size)

16/16 [==============================] - 7s 420ms/step
Accuracy on test dataset:  0.7951318621635437
Classification report:
               precision    recall  f1-score   support

           0       0.79      0.89      0.84       597
           1       0.80      0.65      0.71       389

    accuracy                           0.80       986
   macro avg       0.80      0.77      0.78       986
weighted avg       0.80      0.80      0.79       986

Confusion matrix:
```

[49]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7af90ed1d0c0>

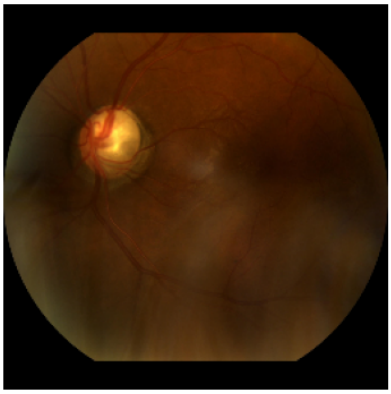


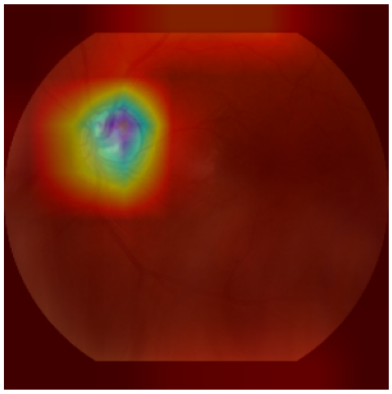

### 0.1.5 Generating the Heatmap

[50]:
```
from cam import GRADgen
res_classifier.load_weights(checkpoint_filepath_res)
for i in range(3):
    random_batch = np.random.randint(0, 10)
    random_index = np.random.randint(0, 32)
    output, image, preds = GRADgen(res_classifier, 'conv5_block3_out',␣
 ↪test_dataset.__getitem__(random_batch)[0][random_index])
    print(f"Predicted label: {preds} \nActual label: {test_dataset.
 ↪__getitem__(random_batch)[1][random_index]}")
    fig = plt.figure(figsize=(15, 15))
    plt.subplot(3, 2, 2*i+1)
    plt.imshow(image)
    plt.axis('off')
    plt.subplot(3, 2, 2*i+2)
    plt.imshow(output)
    plt.axis('off')
```

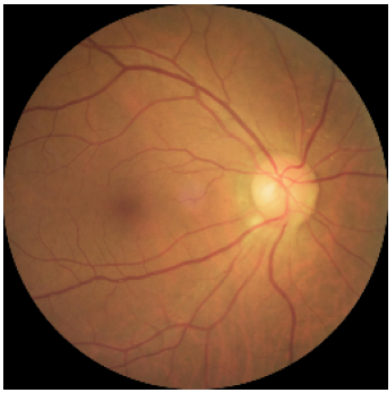
```
    plt.show()
```

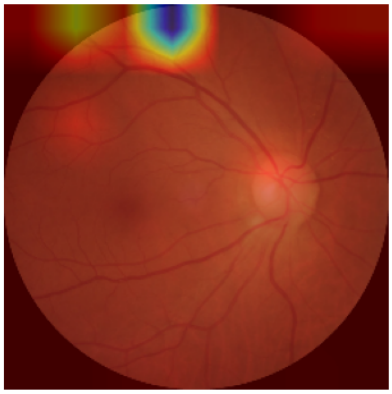
```
1/1 [==============================] - 1s 1s/step
Predicted label: [[1.]]
Actual label: 1
```



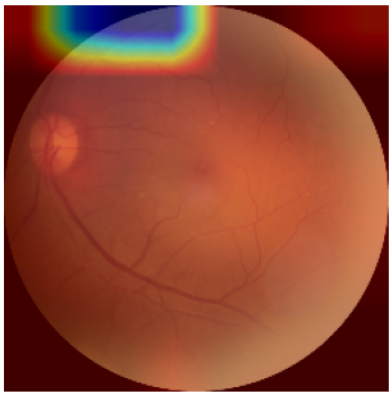


```
1/1 [==============================] - 0s 29ms/step
Predicted label: [[0.]]
Actual label: 0
```





```
1/1 [==============================] - 0s 28ms/step
Predicted label: [[0.]]
Actual label: 0
```



[51]:
```
#wandb.finish()
```

[ ]:
```

```